

POLITECHNIKA WROCŁAWSKA
WYDZIAŁ INFORMATYKI I
TELEKOMUNIKACJI

KIERUNEK: INFORMATYKA TECHNICZNA (ITE)

DOKUMENTACJA Całego projektu

AUTORZY:

Szymon Tyliński
Michał Hendrysiak
Mateusz Brokos

TERMIN:

Poniedziałek, 17:05

PROWADZĄCY:

dr. inż. Marcin Łopuszyński

WROCŁAW, 2025

Table of Contents

Opis wycinka rzeczywistości.....	3
Etap I Modelowanie bazy danych.....	4
UC01 – Zlecenie naprawy roweru	6
UC03 – Rejestracja przyjęcia roweru	7
UC03a – Sprawdzenie dostępnych miejsc w magazynie	7
UC04 – Aktualizacja statusu zlecenia	7
UC04a – Naprawa roweru.....	8
UC04b – Zakończenie naprawy	8
UC04c – Obliczenie kosztu naprawy	8
UC05 – Wykorzystanie części z magazynu	9
UC06 – Aktualizacja stanu magazynu	9
UC07 – Uzupelnienie zapasów magazynu / dodanie nowej części.....	9
UC08 – Ustalenie limitu liczby rowerów	9
UC09 – Przeglądanie historii zleceń	10
Identyfikacja encji	10
Etap II Modelowanie i implementacja bazy danych	11
Wymagania dotyczące dostępu do bazy danych i jej zawartości	11
Diagram logicznego modelu bazy danych	13
Opracowanie diagramu fizycznego modelu bazy	13
Wdrożenie bazy i przetestowanie jej	26
Etap III Modelowanie i implementacja aplikacji.....	28
Opracowanie makiety interfejsu graficznego	28
Opracowanie diagramu klas	36
Aplikacja	36

Lista zmian:

- Poprawiono opis testów jednostkowych.

Opis wycinka rzeczywistości

Serwis rowerowy jest organizacją/obiektem który specjalizuje się w odpłatnym serwisowaniu rowerów. Rowery dostarczone przez klientów i będące ich własnością, są przechowywane w magazynie o ograniczonej przestrzeni. Magazyn dysponuje wydzieloną przestrzenią do magazynowania części zapasowych. Każdego dnia stan magazynu ulega zmianie poprzez wydawanie gotowych rowerów, przyjmowanie rowerów wymagających uwagi serwisu i wymianę części na nowe. Przyjęcie jednośladu następuje poprzez wprowadzenie przez uprawnionego serwisanta danych kontaktowych klienta oraz zakresu świadczonych usług. Każde zlecenie posiada status realizacji oraz unikalny identyfikator.

Sytuacja 1:

Do serwisu przychodzi klient, który składa zlecenie na realizację grupy zabiegów wybranych z dostępnej listy świadczonych usług. Życzenia klienta są zapisane co skutkuje zmniejszeniem liczby dostępnych miejsc w magazynie, przypisaniem statusu oraz unikalnego identyfikatora zlecenia.

Sytuacja 2:

Serwisant przystępuje do realizacji zlecenia co skutkuje zmianą statusu w miarę postępu pracy. W przypadku braku części wstrzymuje zajęcie i zmienia wykonywane zlecenie do czasu uzupełnienia brakującej części.

Sytuacja 3:

Serwisant wykorzystał daną liczbę części zapasowych z magazynu do naprawy rowerów, przez co liczebność tych części spadła poniżej wartości minimalnej. Poskutkuje to wyświetleniem komunikatu o niskim stanie wskazanej części oraz rekomendację uzupełnienia braków.

Sytuacja 4:

Po zakończeniu naprawy, klient odbiera rower z serwisu. W momencie wydania roweru zlecenie zostaje oznaczone jako „zakończone” a liczba dostępnych miejsc w magazynie zwiększa się o jeden. Klient opłaca koszt wykonanych usług i ewentualnie wykorzystanych części zamiennych.

Etap I Modelowanie bazy danych

Zad. 1 Wymagania i użytkownicy systemu

Użytkownicy systemu:

- Serwisant – osoba wykonująca naprawy, przyjmująca zlecenia, zmieniająca status realizacji oraz aktualizująca informacje o wykorzystanych częściach.
- Kierownik serwisu – nadzoruje pracę serwisantów, kontroluje liczbę aktywnych zleceń, ma dostęp do raportów, ustala limit rowerów możliwych do przechowywania.
- Klient – osoba zlecająca usługę serwisową, odbierająca rower po zakończeniu naprawy, dokonująca płatności.

Wymagania funkcjonalne:

A. Obsługa klientów i rowerów

1. Rejestrowanie nowego klienta wraz z danymi kontaktowymi.
2. Przypisywanie do klienta jednego lub więcej rowerów.
3. Zapisywanie danych identyfikacyjnych roweru (marka, model, numer ramy).

B. Obsługa zleceń serwisowych

1. Tworzenie nowego zlecenia serwisowego z informacjami: klient, rower, zakres usług, data przyjęcia, planowany termin, przypisany serwisant.
2. Nadawanie zleceniu statusu: *przyjęte, w naprawie, oczekuje na części, zakończone, odebrane*.
3. Ograniczenie przyjęć nowych zleceń, jeśli liczba rowerów w serwisie osiągnęła ustalony limit.
4. Zmiana statusu zlecenia w miarę postępu prac.
5. Możliwość przeglądania historii zleceń danego klienta.

C. Obsługa magazynu części

9. Ewidencja części zamiennych – nazwa, ilość, cena jednostkowa.
10. Zmniejszanie ilości części po ich wykorzystaniu w ramach zlecenia.
11. Ręczne uzupełnianie ilości części przez osobę uprawnioną.

D. Rozliczenia

12. Obliczanie kosztu naprawy jako sumy kosztu robocizny i wykorzystanych części (bez obsługi płatności i raportów).

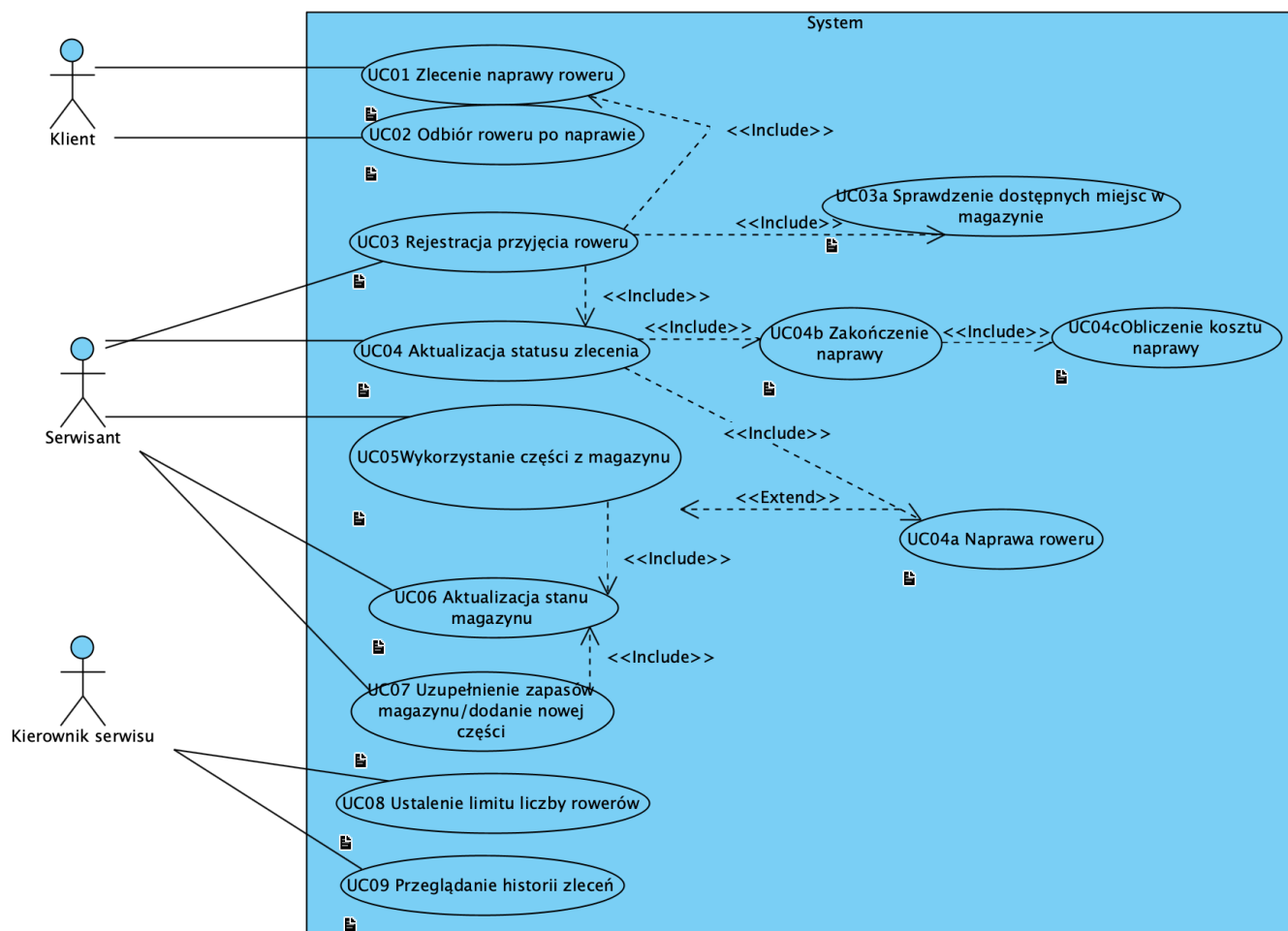
E. Uprawnienia i dostęp

13. Kierownik ustala limit liczby rowerów i ma wgląd do wszystkich zleceń.
14. Serwisant widzi tylko własne zlecenia.
15. Serwisant zarządza danymi o częściach.

Wymagania niefunkcjonalne:

- Spójność danych – dane muszą odzwierciedlać rzeczywisty stan zleceń i magazynu.
- Jednoznaczność identyfikacji – każdy klient, rower, część i zlecenie posiada unikalny identyfikator.
- Integralność referencyjna – nie można usunąć klienta, do którego przypisane są aktywne zlecenia.
- Bezpieczeństwo dostępu – role użytkowników ograniczają dostęp do danych.
- Trwałość danych – wszystkie dane zachowywane są po zakończeniu zleceń.
- Aktualność informacji – zmiany są widoczne natychmiast po wprowadzeniu.
- Niezawodność – system zapobiega błędom logicznym, np. ujemnej liczbie części lub przekroczeniu limitu rowerów.
- Wydajność – operacje na danych muszą być szybkie i niezakłócone.
- Czytelność danych – informacje są przedstawiane w sposób przejrzysty i jednoznaczny.

Zad. 2



UC01 – Zlecenie naprawy roweru

Aktor: Klient

Cel: Zgłoszenie roweru do naprawy

Zdarzenie inicjujące: Klient zgłasza rower do serwisu

Warunki wstępne: W magazynie dostępne jest wolne miejsce na nowy rower

Warunki końcowe: Rower przyjęty do serwisu, utworzone nowe zlecenie

Scenariusz:

1. Klient przekazuje rower i opisuje problem.
2. Serwisant zapisuje dane klienta i rodzaj naprawy.
3. System sprawdza, czy w magazynie jest wolne miejsce.
4. Jeśli tak – system tworzy nowe zlecenie z unikalnym numerem i statusem „przyjęty”.
5. W przeciwnym razie serwisant informuje klienta, że brak miejsca i odmawia przyjęcia roweru.

UC02 – Odbiór roweru po naprawie

Aktor: Klient

Cel: Odebranie naprawionego roweru

Zdarzenie inicjujące: Klient zgłasza się po odbiór roweru

Warunki wstępne: Rower ma status „zakończony”

Warunki końcowe: Rower wydany klientowi, zwolnione miejsce w magazynie

Scenariusz:

1. Klient zgłasza się po odbiór roweru.
2. Serwisant wyszukuje zlecenie w systemie.
3. System potwierdza, że naprawa została zakończona.
4. Serwisant wydaje rower klientowi i oznacza zlecenie jako „zrealizowane”.
5. System zwiększa liczbę dostępnych miejsc w magazynie.

UC03 – Rejestracja przyjęcia roweru

Aktor: Serwisant

Cel: Wprowadzenie nowego zlecenia do systemu

Zdarzenie inicjujące: Przyjęcie roweru od klienta

Warunki wstępne: Klient zgłosił chęć naprawy roweru

Warunki końcowe: Zlecenie utworzone i zapisane

Scenariusz:

1. Serwisant wprowadza dane klienta i opis usterki.
2. System sprawdza dostępne miejsca w magazynie (UC03a).
3. Jeśli miejsce jest dostępne – tworzy wpis zlecenia.
4. System nadaje zleceniu unikalny numer i status „przyjęty”.

UC03a – Sprawdzenie dostępnych miejsc w magazynie

Aktor: System

Cel: Upewnienie się, że serwis ma miejsce na przyjęcie kolejnego roweru

Zdarzenie inicjujące: Próba rejestracji nowego zlecenia

Warunki wstępne: M magazynowy ustalony przez kierownika

Warunki końcowe: Informacja o dostępności miejsca

Scenariusz:

1. System porównuje liczbę aktualnie przechowywanych rowerów z ustalonym limitem.
2. Jeśli limit nie został przekroczony – zwraca informację o wolnym miejscu.
3. Jeśli przekroczony – zwraca komunikat „brak miejsca”.

UC04 – Aktualizacja statusu zlecenia

Aktor: Serwisant

Cel: Zmiana statusu zlecenia w trakcie realizacji

Zdarzenie inicjujące: Zmiana etapu naprawy

Warunki wstępne: Zlecenie istnieje w systemie

Warunki końcowe: Status zlecenia zmieniony na aktualny

Scenariusz:

1. Serwisant wybiera zlecenie z listy.
2. W zależności od etapu pracy aktualizuje status na „w trakcie”, „oczekuje na części”, „zakończony” itp.
3. W przypadku zakończenia naprawy system wykonuje:
 - o UC04a Naprawa roweru
 - o UC04b Zakończenie naprawy
 - o UC04c Obliczenie kosztu naprawy

UC04a – Naprawa roweru

Aktor: Serwisant

Cel: Wykonanie czynności naprawczych

Zdarzenie inicjujące: Zmiana statusu zlecenia na „w trakcie naprawy”

Warunki wstępne: Zlecenie posiada status „przyjęty”

Warunki końcowe: Naprawa wykonana lub wstrzymana

Scenariusz:

1. Serwisant rozpoczyna naprawę.
2. W razie potrzeby pobiera części z magazynu (UC05).
3. System aktualizuje status zlecenia.
4. Po zakończeniu prac serwisant oznacza zlecenie jako „zakończone”.

UC04b – Zakończenie naprawy

Aktor: Serwisant

Cel: Oznaczenie zakończenia zlecenia

Zdarzenie inicjujące: Naprawa została zakończona

Warunki wstępne: Wszystkie prace serwisowe wykonane

Warunki końcowe: Zlecenie ma status „zakończony”

Scenariusz:

1. Serwisant potwierdza zakończenie prac.
2. System zapisuje datę zakończenia.
3. System wylicza koszt naprawy (UC04c).

UC04c – Obliczenie kosztu naprawy

Aktor: System

Cel: Obliczenie końcowego kosztu naprawy

Zdarzenie inicjujące: Zakończenie naprawy

Warunki wstępne: Znane części i czynności wykorzystane w naprawie

Warunki końcowe: Koszt naprawy zapisany w zleceniu

Scenariusz:

1. System sumuje koszt użytych części i roboczogodzin.
2. Wyświetla serwisantowi łączną kwotę dla klienta.
3. Koszt zostaje zapisany w zleceniu.

UC05 – Wykorzystanie części z magazynu

Aktor: Serwisant

Cel: Wydanie części do naprawy

Zdarzenie inicjujące: Naprawa wymaga użycia części

Warunki wstępne: Części dostępne w magazynie

Warunki końcowe: Zaktualizowany stan magazynu

Scenariusz:

1. Serwisant wybiera część potrzebną do naprawy.
2. System zmniejsza liczbę dostępnych egzemplarzy tej części.
3. Jeśli liczba części spadnie poniżej minimum – system zgłasza alert.
4. Serwisant może uzupełnić zapasy (UC07).

UC06 – Aktualizacja stanu magazynu

Aktor: Serwisant

Cel: Utrzymanie aktualnych danych o stanie magazynowym

Zdarzenie inicjujące: Zużycie lub uzupełnienie części

Warunki wstępne: Części zidentyfikowane w systemie

Warunki końcowe: Zaktualizowany stan magazynu

Scenariusz:

1. Serwisant wprowadza zmianę ilości części.

2. System zapisuje nowy stan.
3. W razie potrzeby można dodać nowe części (UC07).

UC07 – Uzupelnienie zapasów magazynu / dodanie nowej części

Aktor: Serwisant

Cel: Uzupelnienie lub dodanie nowych części

Zdarzenie inicjujące: Alert o niskim stanie lub dostawa części

Warunki wstępne: Znany rodzaj części

Warunki końcowe: Części dodane do magazynu

Scenariusz:

1. Serwisant dodaje nową dostawę części.
2. System zwiększa ich liczbę w magazynie lub dodaje nową pozycję.
3. Alert o niskim stanie znika.

UC08 – Ustalenie limitu liczby rowerów

Aktor: Kierownik serwisu

Cel: Określenie maksymalnej liczby rowerów, jakie serwis może przyjąć

Zdarzenie inicjujące: Zmiana warunków pracy lub pojemności magazynu

Warunki wstępne: Kierownik posiada uprawnienia administracyjne

Warunki końcowe: Limit zapisany w systemie

Scenariusz:

1. Kierownik wprowadza nową wartość limitu.
2. System zapisuje ją i stosuje przy przyjmowaniu nowych zleceń.

UC09 – Przeglądanie historii zleceń

Aktor: Kierownik serwisu

Cel: Analiza wykonanych napraw i historii zleceń

Zdarzenie inicjujące: Potrzeba kontroli lub raportu

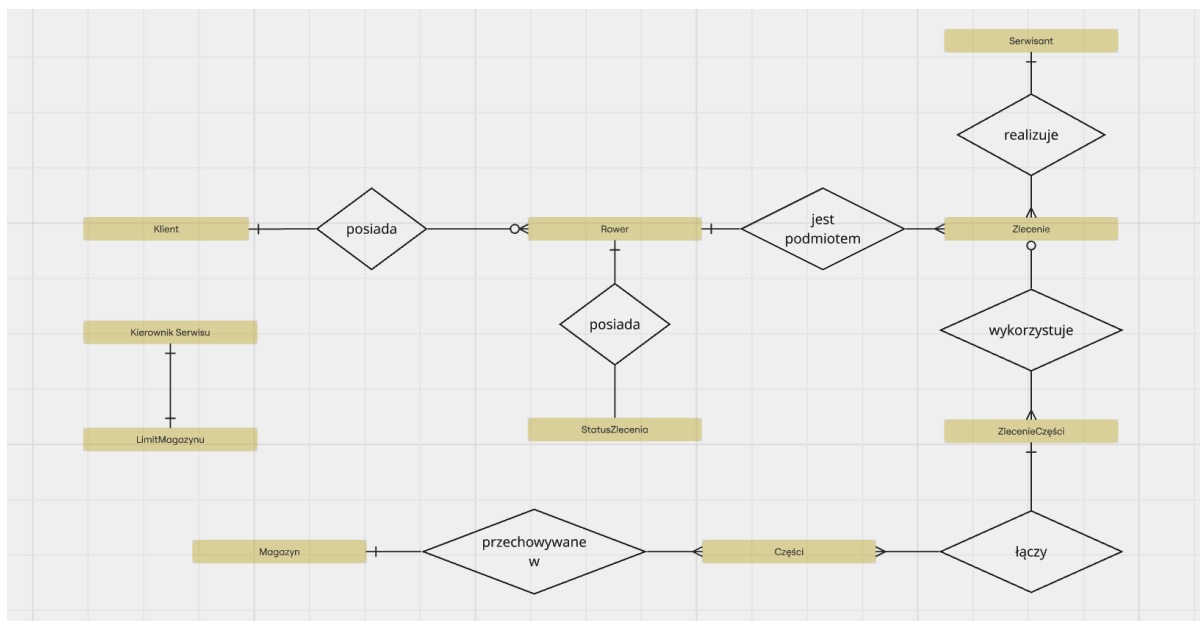
Warunki wstępne: Istnieją zakończone zlecenia, w przypadku braku zakończonych zleceń otrzymanie powiadomienia o tym zdarzeniu.

Warunki końcowe: Kierownik otrzymuje zestawienie danych

Scenariusz:

1. Kierownik wybiera zakres dat lub kryteria filtrowania.
2. System wyświetla listę zakończonych zleceń wraz z informacjami o kosztach i częściach.
3. Kierownik może przeglądać szczegóły wybranego zlecenia.

Identyfikacja encji



Opis:

LimitMagazynu – zbiór maksymalnych oraz minimalnych przedziałów liczebności części w magazynie ustalone przez kierownika

ZlecenieCzęści – zawiera dane o zleceniach oraz wykorzystanych w nich części

Etap II Modelowanie i implementacja bazy danych

Wymagania dotyczące dostępu do bazy danych i jej zawartości

- a. Role użytkowników
 - i. Kierownik – nadzoruje serwis, ma dostęp do wszystkich danych, może ustalać limit magazynu.
 - ii. Serwisant – obsługuje klientów, przyjmuje rowery, tworzy i aktualizuje zlecenia, zarządza częściami i magazynem.
 - iii. Aplikacja – konto techniczne używane przez warstwę aplikacji do obsługi standardowych operacji na bazie.
 - iv. Administrator bazy – pełne uprawnienia, zarządza strukturą bazy, uprawnieniami i kopią zapasową.
- b. Zakres danych
 - i. Klient - Przechowuje dane identyfikujące klienta, będącego właścicielem rowerów.

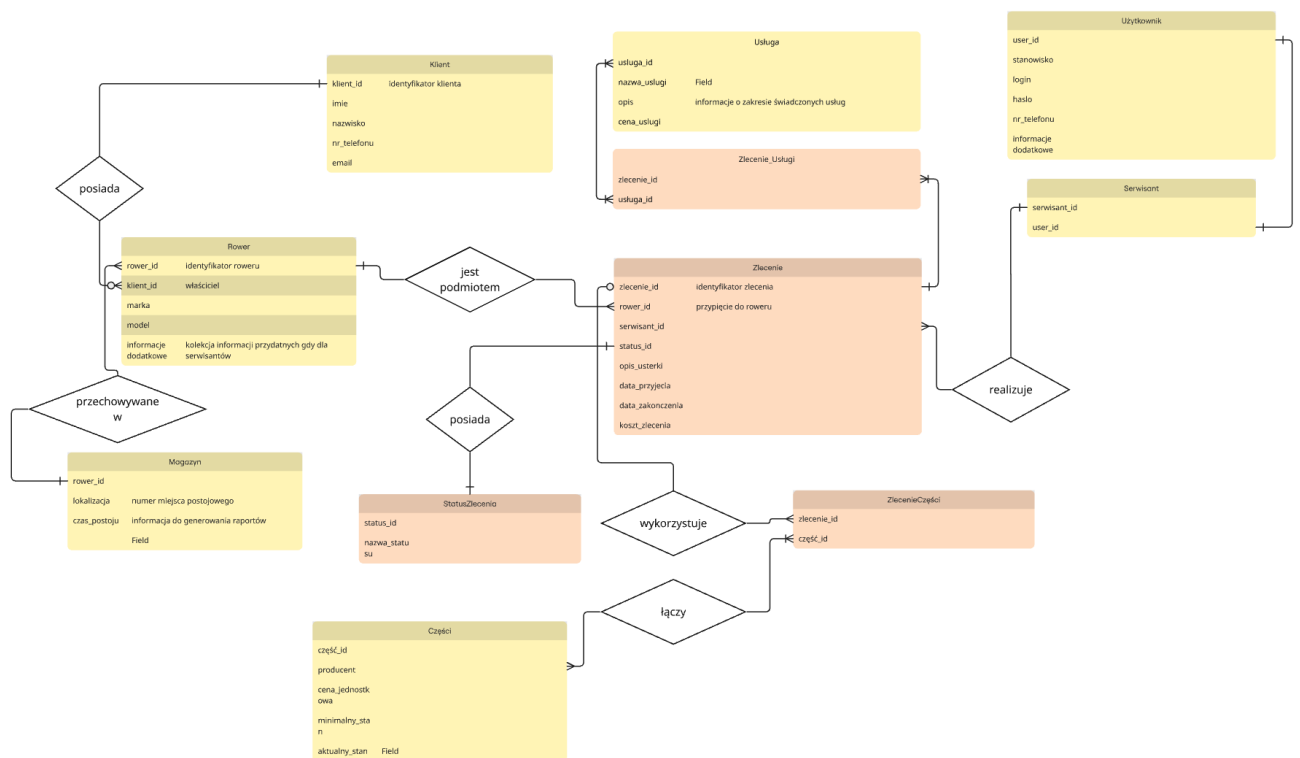
- ii. Rower - Rejestr rowerów należących do klientów, z podstawowymi informacjami technicznymi.
- iii. Zlecenie - Reprezentuje zgłoszenie serwisowe dotyczące danego roweru, z informacją o usterce, statusie, serwisancie i koszcie.
- iv. Serwisant - Powiązany z tabelą użytkowników. Określa, którzy użytkownicy mają rolę serwisanta i mogą wykonywać naprawy.
- v. Użytkownik - Jednolita tabela zawierająca wszystkich użytkowników systemu — zarówno serwisantów, jak i kierowników.
- vi. Usługa - Zawiera listę usług serwisowych, ich nazwy, opisy oraz ceny.
- vii. Zlecenie_usługi - Tabela pośrednicząca N:M między zleceniami a usługami. Informuje, jakie usługi wykonano w ramach zlecenia.
- viii. Część - Informacje o częściach serwisowych, ich koszcie jednostkowym, minimalnym i aktualnym stanie magazynowym.
- ix. Zlecenie_części - Tabela łącząca zlecenia z częściami zużytymi przy naprawie.
- x. Status_zlecenia - Słownik statusów opisujących aktualny etap realizacji zgłoszenia.
- xi. Magazyn - Informacje o przechowywanych rowerach: miejsce postojowe i czas oczekiwania na serwis.

c. Uprawnienia użytkowników do tabeli

Rola / Tabela	klie nt	row er	zlece nie	serwis ant	użytkowni k	usłu ga	zlecenie_usł ugi	czę ść	zlecenie_czę ści	status_zlece nia	magaz yn
Kierownik	S	S	S	S	S, I, U	S	S	S	S	S	S
Serwisant	S, I	S, I	S, I, U	S	S	S	S, I, U	S, I, U	I, U	S	S, U
Aplikacja	S, I	S, I	S, I, U	S	S	S, I	S, I, U	S, I	S, I, U	S, I, U	S, I
Administrator	I, S, U, D	I, S, U, D	I, S, U, D	I, S, U, D	I, S, U, D	I, S, U, D	I, S, U, D	I, S, U, D	I, S, U, D	I, S, U, D	I, S, U, D

Legenda: I – insert; S – select; U – update; D - delete

Diagram logicznego modelu bazy danych



Opracowanie diagramu fizycznego modelu bazy

- d. Zdecydowaliśmy się na utworzenie bazy danych na serwerze PostgreSQL z uwagi na łatwą obsługę w systemie MacOS oraz fakt, że jest ogólnie dostępny.
- e. Utworzyliśmy bazę danych

```

szymon@MacBook-Air-Szymon-2 ~ % brew services start postgresql
==> Successfully started `postgresql@14` (label: homebrew.mxcl.postgresql@14)
szymon@MacBook-Air-Szymon-2 ~ % createdb serwis_rowerowy
szymon@MacBook-Air-Szymon-2 ~ % psql serwis_rowerowy
psql (14.20 (Homebrew))
Type "help" for help.

serwis_rowerowy=#
    
```

- f. Utworzyliśmy skrypt SQL implementujący graf logiczny bazy danych:

Tworzenie tabel:

```
-- =====
-- UŻYTKOWNICY SYSTEMU
-- =====

CREATE TABLE Uzytkownik (
    user_id SERIAL PRIMARY KEY,
    stanowisko VARCHAR(50) NOT NULL CHECK (stanowisko IN ('Administrator', 'Serwisant', 'Kierownik')),
    login VARCHAR(50) UNIQUE NOT NULL,
    haslo VARCHAR(255) NOT NULL,
    nr_telefonu VARCHAR(20) CHECK (nr_telefonu ~ '^[0-9]{9,15}$'),
    informacje_dodatkowe TEXT
);

CREATE TABLE Serwisant (
    serwisant_id SERIAL PRIMARY KEY,
    user_id INT NOT NULL UNIQUE REFERENCES Uzytkownik(user_id) ON DELETE CASCADE
);

-- =====
-- KLIENT / ROWER
-- =====

CREATE TABLE Klient (
    klient_id SERIAL PRIMARY KEY,
    imie VARCHAR(50) NOT NULL,
    nazwisko VARCHAR(50) NOT NULL,
    nr_telefonu VARCHAR(20) CHECK (nr_telefonu ~ '^[0-9]{9,15}$'),
    email VARCHAR(100) CHECK (email ~ '^([A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,})$')
);

CREATE TABLE Rower (
    rower_id SERIAL PRIMARY KEY,
    klient_id INT NOT NULL REFERENCES Klient(klient_id) ON DELETE CASCADE,
```

```

marka VARCHAR(50) NOT NULL,

model VARCHAR(50) NOT NULL,

informacje_dodatkowe TEXT
);

-- =====
-- MAGAZYN
-- =====

CREATE TABLE Magazyn (

    rower_id INT PRIMARY KEY REFERENCES Rower(rower_id) ON DELETE CASCADE,

    lokalizacja VARCHAR(50) NOT NULL,

    czas_postoju INTERVAL
);

-- TRIGGER: usunięcie roweru → automatyczne usunięcie wpisu z magazynu
CREATE OR REPLACE FUNCTION usun_z_magazynu()

RETURNS TRIGGER AS $$

BEGIN

    DELETE FROM Magazyn WHERE rower_id = OLD.rower_id;

    RETURN OLD;

END;

$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_usun_z_magazynu

AFTER DELETE ON Rower

FOR EACH ROW EXECUTE FUNCTION usun_z_magazynu();

-- =====
-- STATUS ZLECENIA
-- =====

CREATE TABLE StatusZlecenia (

    status_id SERIAL PRIMARY KEY,

```

```

nazwa_statusu VARCHAR(50) UNIQUE NOT NULL
);

-- =====
-- ZLECENIE
-- =====

CREATE TABLE Zlecenie (
    zlecenie_id SERIAL PRIMARY KEY,
    rower_id INT NOT NULL REFERENCES Rower(rower_id) ON DELETE CASCADE,
    serwisant_id INT REFERENCES Serwisant(serwisant_id) ON DELETE SET NULL,
    status_id INT NOT NULL REFERENCES StatusZlecenia(status_id),
    opis_usterki TEXT NOT NULL,
    data_przyjecia DATE NOT NULL,
    data_zakonczenia DATE,
    koszt_zlecenia NUMERIC(10,2) DEFAULT 0
);

-- =====
-- USŁUGI
-- =====

CREATE TABLE Usługa (
    usługa_id SERIAL PRIMARY KEY,
    nazwa_usługi VARCHAR(100) NOT NULL,
    opis TEXT,
    cena_usługi NUMERIC(10,2) NOT NULL
);

CREATE TABLE Zlecenie_Usługi (
    zlecenie_id INT REFERENCES Zlecenie(zlecenie_id) ON DELETE CASCADE,
    usługa_id INT REFERENCES Usługa(usługa_id),
    PRIMARY KEY (zlecenie_id, usługa_id)
);

```

```

-- =====
-- CZĘŚCI
-- =====

CREATE TABLE Czesc (
    czesc_id SERIAL PRIMARY KEY,
    producent VARCHAR(100),
    cena_jednostkowa NUMERIC(10,2),
    minimalny_stan INT NOT NULL,
    aktualny_stan INT NOT NULL
);

CREATE TABLE ZlecenieCzesci (
    zlecenie_id INT REFERENCES Zlecenie(zlecenie_id) ON DELETE CASCADE,
    czesc_id INT REFERENCES Czesc(czesc_id),
    ilosc INT NOT NULL CHECK (ilosc > 0),
    PRIMARY KEY (zlecenie_id, czesc_id)
);

```

Procedury:

```

-----
-- UŻYTKOWNICY
-----

-- dodawanie użytkownika

CREATE OR REPLACE FUNCTION dodaj_uzytkownika(
    p_stanowisko VARCHAR,
    p_login VARCHAR,
    p_haslo VARCHAR,

```

```

    p_nr_tel VARCHAR,

    p_info TEXT

) RETURNS INT AS $$

DECLARE

    nowy_id INT;

BEGIN

    INSERT INTO Uzytkownik(stanowisko, login, haslo, nr_telefonu, informacje_dodatkowe)

    VALUES (p_stanowisko, p_login, p_haslo, p_nr_tel, p_info)

    RETURNING user_id INTO nowy_id;

    RETURN nowy_id;

END;

$$ LANGUAGE plpgsql;

-- dodanie serwisanta (użytkownik musi mieć stanowisko = 'Serwisant')

CREATE OR REPLACE FUNCTION dodaj_serwisanta(p_user_id INT) RETURNS INT AS $$

DECLARE

    nowy_serwisant INT;

BEGIN

    INSERT INTO Serwisant(user_id)

    VALUES (p_user_id)

    RETURNING serwisant_id INTO nowy_serwisant;

    RETURN nowy_serwisant;

END;

$$ LANGUAGE plpgsql;

-- aktualizacja danych użytkownika

CREATE OR REPLACE PROCEDURE aktualizuj_uzytkownika(

    p_user_id INT,

    p_stanowisko VARCHAR,

    p_nr_tel VARCHAR,

```

```

    p_info TEXT
) AS $$
BEGIN
    UPDATE Uzytkownik
    SET stanowisko = p_stanowisko,
        nr_telefonu = p_nr_tel,
        informacje_dodatkowe = p_info
    WHERE user_id = p_user_id;
END;
$$ LANGUAGE plpgsql;

-- usuwanie uzytkownika
CREATE OR REPLACE PROCEDURE usun_uzytkownika(p_user_id INT) AS $$
BEGIN
    DELETE FROM Uzytkownik
    WHERE user_id = p_user_id;
END;
$$ LANGUAGE plpgsql;

-- zmiana hasla
CREATE OR REPLACE PROCEDURE zmien_haslo(
    p_user_id INT,
    p_nowe_haslo VARCHAR
) AS $$
BEGIN
    UPDATE Uzytkownik
    SET haslo = p_nowe_haslo
    WHERE user_id = p_user_id;
END;
$$ LANGUAGE plpgsql;

```

```
-- KLIENT / ROWER / MAGAZYN
```

```
-----
```

```
-- dodawanie klienta
```

```
CREATE OR REPLACE FUNCTION dodaj_klienta(
```

```
    p_imie VARCHAR,
```

```
    p_nazwisko VARCHAR,
```

```
    p_nr_tel VARCHAR,
```

```
    p_email VARCHAR
```

```
) RETURNS INT AS $$
```

```
DECLARE
```

```
    nowy_id INT;
```

```
BEGIN
```

```
    INSERT INTO Klient(imie, nazwisko, nr_telefonu, email)
```

```
    VALUES (p_imie, p_nazwisko, p_nr_tel, p_email)
```

```
    RETURNING klient_id INTO nowy_id;
```

```
    RETURN nowy_id;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
-- dodawanie roweru
```

```
CREATE OR REPLACE FUNCTION dodaj_rower(
```

```
    p_klient_id INT,
```

```
    p_marka VARCHAR,
```

```
    p_model VARCHAR,
```

```
    p_info TEXT
```

```
) RETURNS INT AS $$
```

```
DECLARE
```

```
    nowy_id INT;
```

```
BEGIN
```

```
    INSERT INTO Rower(klient_id, marka, model, informacje_dodatkowe)
```

```

VALUES (p_klient_id, p_marka, p_model, p_info)

RETURNING rower_id INTO nowy_id;

RETURN nowy_id;
END;
$$ LANGUAGE plpgsql;

-- przyjęcie roweru do magazynu

CREATE OR REPLACE PROCEDURE przyjmij_do_magazynu(
    p_rower_id INT,
    p_lokalizacja VARCHAR
) AS $$
BEGIN
    INSERT INTO Magazyn(rower_id, lokalizacja, czas_postoju)
    VALUES (p_rower_id, p_lokalizacja, NOW() - NOW()); -- czas zerowy
END;
$$ LANGUAGE plpgsql;

-- wydanie roweru z magazynu

CREATE OR REPLACE PROCEDURE wydaj_z_magazynu(p_rower_id INT) AS $$
BEGIN
    DELETE FROM Magazyn
    WHERE rower_id = p_rower_id;
END;
$$ LANGUAGE plpgsql;

-- aktualizacja lokalizacji roweru

CREATE OR REPLACE PROCEDURE zmien_lokalizacje(
    p_rower_id INT,
    p_lokalizacja VARCHAR
) AS $$
BEGIN

```

```

UPDATE Magazyn

SET lokalizacja = p_lokalizacja

WHERE rower_id = p_rower_id;

END;

$$ LANGUAGE plpgsql;

-----

-- ZLECENIA

-----

-- tworzenie zlecenia

CREATE OR REPLACE FUNCTION utworz_zlecenie(

    p_rower_id INT,

    p_opis TEXT

) RETURNS INT AS $$

DECLARE

    nowy_id INT;

    id_statusu INT;

BEGIN

    SELECT status_id INTO id_statusu

    FROM StatusZlecenia

    WHERE nazwa_statusu = 'Przyjęte';

    INSERT INTO Zlecenie(rower_id, status_id, opis_usterki, data_przyjecia)

    VALUES (p_rower_id, id_statusu, p_opis, CURRENT_DATE)

    RETURNING zlecenie_id INTO nowy_id;

    RETURN nowy_id;

END;

$$ LANGUAGE plpgsql;

-- przypisanie serwisanta

```

```

CREATE OR REPLACE PROCEDURE przypisz_serwisanta(
    p_zlecenie_id INT,
    p_serwisant_id INT
) AS $$
BEGIN
    UPDATE Zlecenie
    SET serwisant_id = p_serwisant_id
    WHERE zlecenie_id = p_zlecenie_id;
END;
$$ LANGUAGE plpgsql;

-- zmiana statusu zlecenia

CREATE OR REPLACE PROCEDURE zmien_status(
    p_zlecenie_id INT,
    p_status VARCHAR
) AS $$
DECLARE
    id_statusu INT;
BEGIN
    SELECT status_id INTO id_statusu
    FROM StatusZlecenia
    WHERE nazwa_statusu = p_status;

    UPDATE Zlecenie
    SET status_id = id_statusu
    WHERE zlecenie_id = p_zlecenie_id;
END;
$$ LANGUAGE plpgsql;

-- dodanie usługi

CREATE OR REPLACE PROCEDURE dodaj_usluge_do_zlecenia(
    p_zlecenie_id INT,

```

```

    p_usluga_id INT
) AS $$
BEGIN
    INSERT INTO Zlecenie_Uslugi(zlecenie_id, usluga_id)
    VALUES (p_zlecenie_id, p_usluga_id);
END;
$$ LANGUAGE plpgsql;

-- dodanie części
CREATE OR REPLACE PROCEDURE dodaj_czesc_do_zlecenia(
    p_zlecenie_id INT,
    p_czesc_id INT,
    p_ilosc INT
) AS $$
BEGIN
    INSERT INTO ZlecenieCzesci(zlecenie_id, czesc_id, ilosc)
    VALUES (p_zlecenie_id, p_czesc_id, p_ilosc);
END;
$$ LANGUAGE plpgsql;

-- automatyczne przeliczenie kosztu
CREATE OR REPLACE PROCEDURE przelicz_koszt(p_zlecenie_id INT) AS $$
DECLARE
    wart_uslug NUMERIC := 0;
    wart_czesci NUMERIC := 0;
BEGIN
    SELECT COALESCE(SUM(U.cena_uslugi), 0)
    INTO wart_uslug
    FROM Zlecenie_Uslugi ZU
    JOIN Usluga U ON ZU.usluga_id = U.usluga_id
    WHERE ZU.zlecenie_id = p_zlecenie_id;

```

```

SELECT COALESCE(SUM(C.cena_jednostkowa * ZC.ilosc), 0)
INTO wart_czesci
FROM ZlecenieCzesci ZC
JOIN Czesc C ON ZC.czesc_id = C.czesc_id
WHERE ZC.zlecenie_id = p_zlecenie_id;

UPDATE Zlecenie
SET koszt_zlecenia = wart_uslug + wart_czesci
WHERE zlecenie_id = p_zlecenie_id;
END;
$$ LANGUAGE plpgsql;

-- zamknięcie zlecenia
CREATE OR REPLACE PROCEDURE zamknij_zlecenie(p_zlecenie_id INT) AS $$
DECLARE
    id_statusu INT;
BEGIN
    CALL przelicz_koszt(p_zlecenie_id);

    SELECT status_id INTO id_statusu
    FROM StatusZlecenia
    WHERE nazwa_statusu = 'Zakończone';

    UPDATE Zlecenie
    SET status_id = id_statusu,
        data_zakonczenia = CURRENT_DATE
    WHERE zlecenie_id = p_zlecenie_id;
END;
$$ LANGUAGE plpgsql;

```

Wdrożenie bazy i przetestowanie jej

```
CREATE TABLE
CREATE TABLE
ERROR: relation "klient" already exists
ERROR: relation "rower" already exists
ERROR: relation "magazyn" already exists
CREATE FUNCTION
CREATE TRIGGER
ERROR: relation "statuszlecenia" already exists
CREATE TABLE
ERROR: relation "usluga" already exists
CREATE TABLE
ERROR: relation "czesc" already exists
CREATE TABLE
serwis_rowerowy=#
```

```
UPDATE Zlecenie
SET status_id = id_statusu,
    data_zakonczenia = CURRENT_DATE
WHERE zlecenie_id = p_zlecenie_id;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
CREATE FUNCTION
CREATE PROCEDURE
CREATE PROCEDURE
CREATE PROCEDURE
CREATE FUNCTION
CREATE FUNCTION
CREATE PROCEDURE
CREATE PROCEDURE
CREATE PROCEDURE
CREATE PROCEDURE
CREATE FUNCTION
CREATE PROCEDURE
CREATE PROCEDURE
CREATE PROCEDURE
CREATE PROCEDURE
CREATE PROCEDURE
CREATE PROCEDURE
serwis_rowerowy=#
```

Tworzenie usług i części

```
serwis_rowerowy=# INSERT INTO Usługa (nazwa_uslugi, opis, cena_uslugi)
VALUES
('Przegląd podstawowy', 'Kontrola hamulców, przerzutek i opon', 120),
('Wymiana opon', 'Zmiana opon przedniej i tylnej', 80);
INSERT INTO Czesc (producent, cena_jednostkowa, minimalny_stan, aktualny_stan)
VALUES
('Shimano', 25, 2, 10),
('KMC', 60, 1, 5);
INSERT 0 2
INSERT 0 2
serwis_rowerowy=#
```

Dodanie klienta

```
serwis_rowerowy=# SELECT dodaj_klienta(
'Jan',
'Kowalski',
'123456789',
'jan.kowalski@example.com'
) AS nowy_klient;
-- Pobranie ID klienta
SELECT klient_id FROM Klient ORDER BY klient_id DESC LIMIT 1;
      nowy_klient
-----
              1
(1 row)

      klient_id
-----
              1
(1 row)

serwis_rowerowy=#
```

Dodanie roweru

```
serwis_rowerowy=# SELECT dodaj_rower(
1, -- klient_id
'Kross',
'Hexagon X5',
'Rower górski do serwisu testowego'
) AS rower_id;
      rower_id
-----
              1
(1 row)

serwis_rowerowy=#
```

Przyjęcie do magazynu

```
serwis_rowerowy=# CALL przyjmij_do_magazynu(1, 'Stojak A2');
CALL
serwis_rowerowy=#
```

Utworzenie zlecenia

```
serwis_rowerowy=# SELECT utworz_zlecenie(  
1, -- rower_id  
'Hamulec piszczy i słabo hamuje'  
) AS nowe_zlecenie;  
-- Pobranie ID zlecenia  
SELECT zlecenie_id FROM Zlecenie ORDER BY zlecenie_id DESC LIMIT 1;  
nowe_zlecenie  
-----  
2  
(1 row)  
  
zlecenie_id  
-----  
2  
(1 row)  
  
serwis_rowerowy=#
```

Etap III Modelowanie i implementacja aplikacji

Opracowanie makiety interfejsu graficznego

Ekran logowania do systemu

Prezentowany ekran stanowi punkt wejściowy do aplikacji. Umożliwia uwierzytelnienie użytkownika na podstawie loginu i hasła. System weryfikuje wprowadzone dane z tabelą Uzytkownik w bazie danych. Na podstawie przypisanej roli (zdefiniowanej w kolumnie stanowisko), aplikacja przekierowuje użytkownika do odpowiedniego panelu: Panelu Serwisanta (obsługa zleceń) lub Panelu Kierownika (raporty i zarządzanie).

System Serwisu Rowerowego - Logowanie

Login:	admin
Hasło:	****
Zaloguj się	

Panel główny Serwisanta – Lista zleceń

Widok ten jest głównym narzędziem pracy serwisanta. Centralną część zajmuje tabela prezentująca listę aktywnych zleceń, pobieranych z widoku bazy danych łączącego tabele Zlecenie, Rower i Klient. Kolumna „Status” pozwala na szybką identyfikację etapu prac (np. „Przyjęte”, „W naprawie”). Interfejs zapewnia szybki

dostęp do kluczowych funkcji: rejestracji nowego zlecenia, przejścia do magazynu części oraz edycji wybranych napraw.

Panel Serwisanta - Lista Zleceń

Dzień dobry, Jan Kowalski (Serwisant) [Wyloguj]					
Lista Zleceń Magazyn Klienci					
+ Nowe Zlecenie (Przyjęcie roweru)					
ID	Data	Rower	Klient	Status	Akcja
201	2025-01-02	Kross Hexagon	A. Nowak	Przyjęte	Edytuj
200	2025-01-01	Trek Marlin	B. Wiśniewski	W naprawie	Edytuj
199	2024-12-30	Giant Talon	C. Zielińska	Oczekuje na części	Edytuj

Formularz rejestracji nowego zlecenia (Przyjęcie roweru)

Makieta przedstawia formularz realizujący przypadek użycia **UC03 (Rejestracja przyjęcia roweru)**. Interfejs został zaprojektowany tak, aby w jednym kroku obsłużyć logicznie powiązane operacje bazodanowe: wyszukanie lub dodanie klienta, identyfikację roweru oraz – co kluczowe – przypisanie lokalizacji magazynowej. Pole „Lokalizacja” jest wymagane przez procedurę składowaną `przyjmij_do_magazynu`, a wskaźnik dostępności miejsca weryfikuje limit magazynowy przed zatwierdzeniem formularza.

Edycja Zlecenia #201

Status: W TRAKCIE

Rower: Kross Hexagon (A. Nowak)

Usterka: "Piszcza hamulce..."

Nazwa	Ilość	Cena/szt	Suma
Klocki Hamulcowe	2	25.00	50.00
Dętka 29"	1	20.00	20.00
+ Dodaj część z magazynu			

Nazwa Usługi	Cena
Regulacja hamulców	40.00
Wymiana dętki	30.00
+ Dodaj usługę	

Zapisz zmiany

Zakończ naprawę (Rozlicz)

Ekran szczegółów i realizacji naprawy

Widok edycji konkretnego zlecenia umożliwia serwisantowi ewidencję wykonanych prac. Sekcje „Wykorzystane Części” oraz „Wykonane Usługi” odpowiadają odpowiednio tabelom łączącym Zlecenie_Czesci i Zlecenie_Uslugi. Przycisk „Zakończ naprawę” inicjuje sekwencję zamykającą zlecenie: walidację wprowadzonych danych, wywołanie procedury przelicz_koszt (sumującej koszt materiałów i robocizny) oraz aktualizację statusu na „Zakończone”.

Formularz: Przyjęcie Roweru (Nowe Zlecenie)

1. Dane Klienta		
Telefon:	500 123 456	Szukaj
Imie:	Anna	
Nazwisko:	Nowak	
2. Dane Roweru		
Marka:	Kross	
Model:	Hexagon X5	
Info dodatkowe:	Rysy na ramie	
3. Szczegóły Zlecenia		
Opis usterki:	Piszcza hamulce, wymiana dętki	
4. Magazyn (Wymagane!)		
Dostępność miejsca: TAK (Wolnych: 5)	v	
Lokalizacja (Wieszak):	B-12	
Anuluj >> PRZYJMIJ DO SERWISU <<		

1. Diagramy czynności

Diagram czynności dla procesu logowania i autoryzacji

Diagram przedstawia mechanizm weryfikacji tożsamości użytkownika podczas uruchamiania aplikacji. System wykonuje zapytanie SELECT do tabeli Użytkownik, sprawdzając zgodność loginu i hasła. Po pomyślnej autoryzacji następuje rozgałęzienie ścieżki sterowania w zależności od pola stanowisko. Mechanizm ten zapewnia separację uprawnień, gwarantując, że funkcje administracyjne (dostępne dla Kierownika) są niedostępne dla konta Serwisanta.

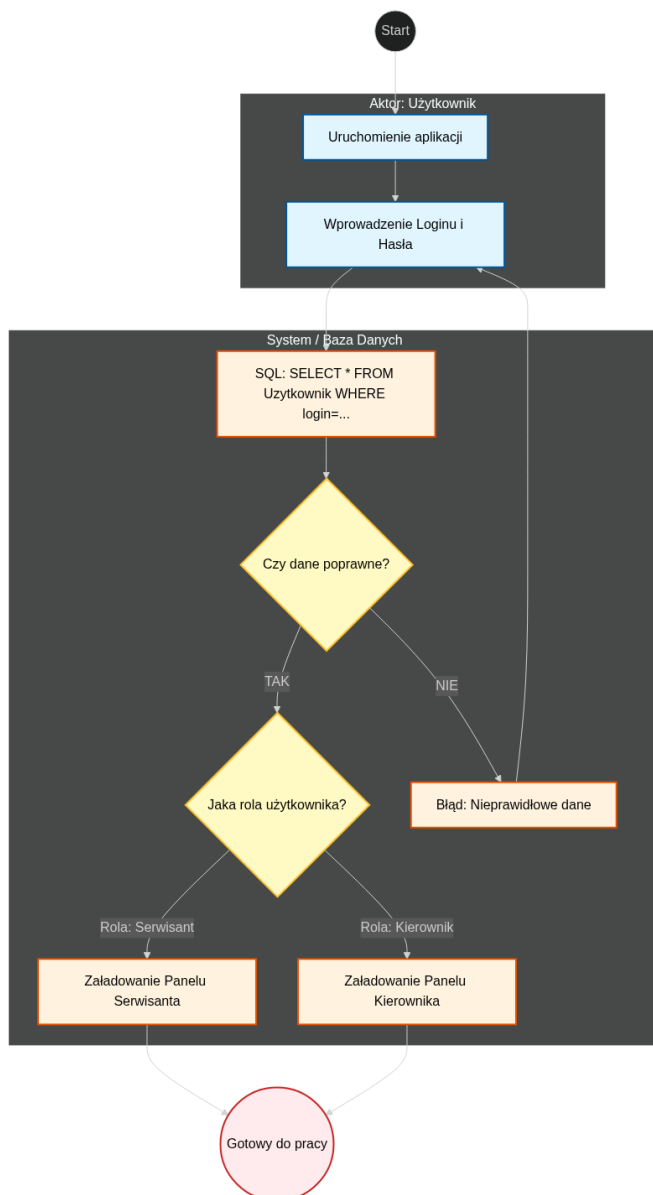


Diagram czynności dla procesu rejestracji przyjęcia roweru (UC03)

Diagram ilustruje algorytm przyjmowania nowego zlecenia serwisowego. Proces rozpoczyna się od weryfikacji istnienia klienta i roweru w bazie danych – w przypadku braku rekordów, system wymusza ich utworzenie (funkcje `dodaj_klienta`, `dodaj_rower`). Kluczowym elementem decyzyjnym jest sprawdzenie dostępności miejsca w magazynie. Jeśli limit nie został przekroczony, system wywołuje procedurę `przyjmij_do_magazynu` oraz tworzy nowe zlecenie ze statusem początkowym. W przeciwnym razie proces kończy się odmową przyjęcia.

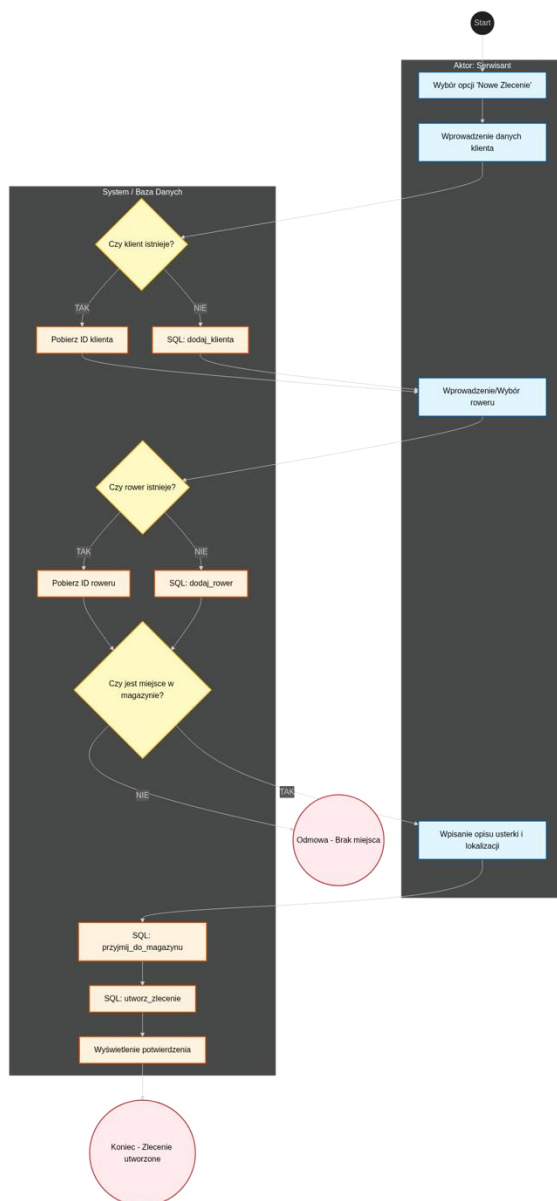


Diagram czynności dla procesu zakończenia naprawy (UC04)

Schemat przedstawia logikę finalizacji prac serwisowych. Aplikacja weryfikuje, czy serwisant wprowadził wszystkie zużyte części. Po zatwierdzeniu przez użytkownika, system uruchamia procedurę składowaną zamknij_zlecenie. Procedura ta automatycznie oblicza całkowity koszt naprawy (agregując ceny usług i części), ustawia datę zakończenia na bieżącą oraz zmienia status zlecenia, co jest sygnałem gotowości roweru do odbioru.

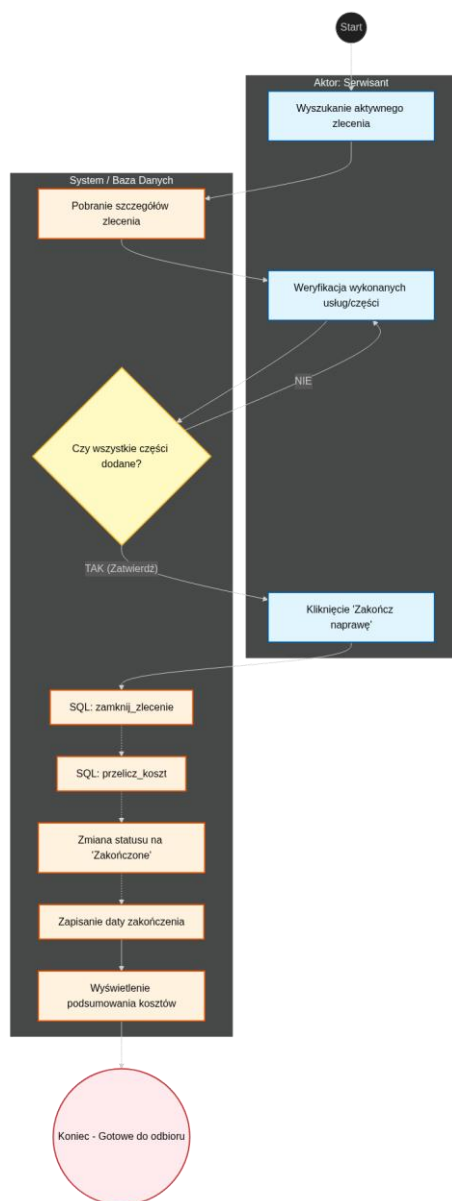
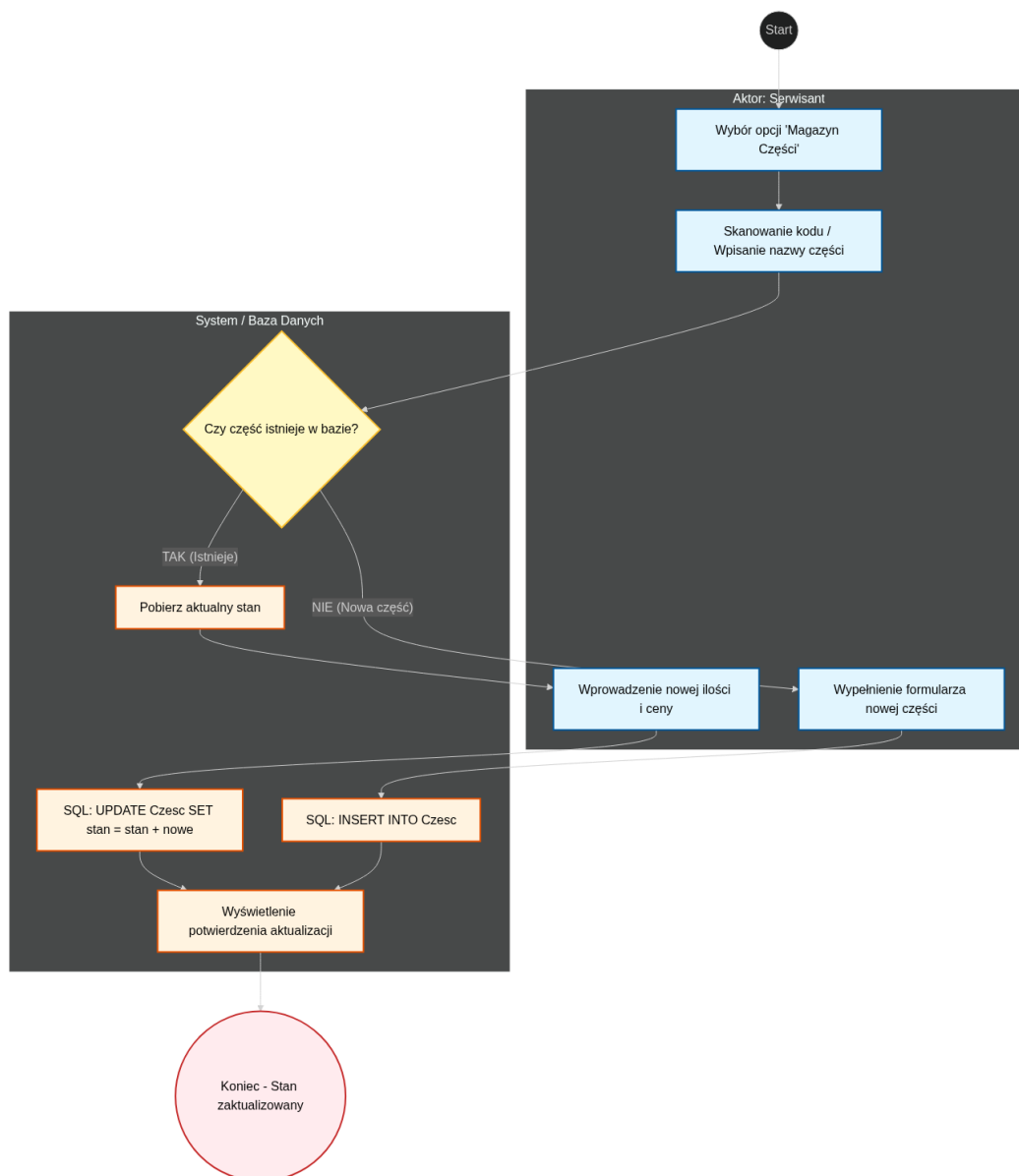


Diagram czynności dla zarządzania stanem magazynowym (UC07)

Diagram obrazuje proces aktualizacji stanów magazynowych części zamiennych. Algorytm sprawdza, czy skanowana lub wprowadzana część istnieje już w tabeli Czesc. W przypadku istniejącego asortymentu następuje aktualizacja (UPDATE) stanu ilościowego. Dla nowych towarów system wymusza wypełnienie pełnego

formularza (producent, cena) i wykonuje operację wstawienia nowego rekordu (INSERT), zapewniając spójność inwentarza.



Opracowanie diagramu klas

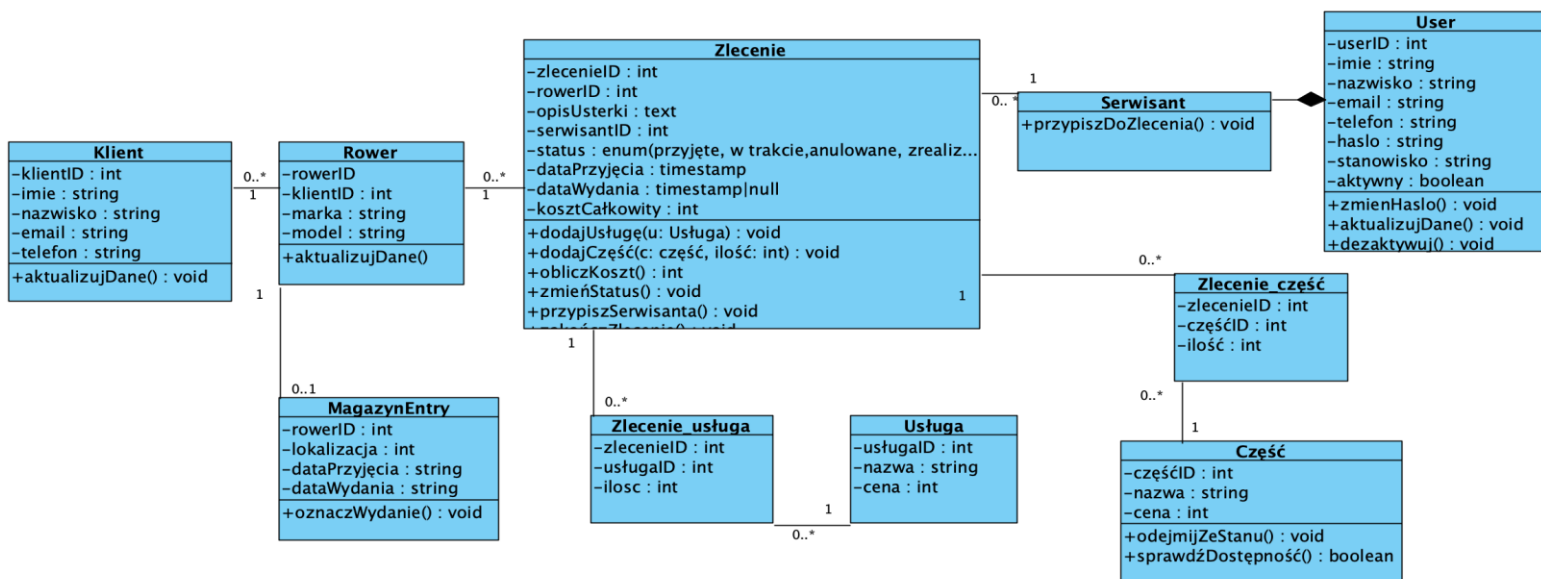


Figure 1 -diagram klas i ich powiązania w aplikacji bazodanowej

Aplikacja

Aplikacja została napisana w języku Java i służy do obsługi danych przechowywanych w bazie danych. Komunikacja z bazą realizowana jest za pomocą JDBC, co umożliwia wykonywanie zapytań SQL oraz zarządzanie danymi.

Aplikacja umożliwia wykonywanie podstawowych operacji danych, takich jak:

- Dodawanie nowych rekordów
- Edycja istniejących rekordów
- Usuwanie rekordów
- Wyświetlanie danych z bazy

Kod aplikacji został podzielony na logiczne części odpowiedzialne za różne aspekty jej działania, gdzie jedna część odpowiedzialna jest za wyświetlanie danych oraz interakcję z użytkownikiem, a druga odpowiada za logikę wywołań i komunikację z bazą danych.

Testy aplikacji

a) Intuicyjność i “idioto-odporność” aplikacji

Wszystkie pola tekstowe z możliwością wprowadzanie danych zostały zabezpieczone i przetestowane przed wprowadzeniem niepoprawnych danych, np. wprowadzenie liter w numerze telefonu

Cel testu: Weryfikacja odporności formularza “Dodaj użytkownika” na błędne dane wejściowe

Scenariusz: Próba wprowadzenia znaków alfabetycznych “abcd” w pole przeznaczone na numer telefonu

Wynik: Aplikacja rozpoznała niezgodność typu danych i wyświetliła czytelny komunikat “Niepoprawne dane!”

Zastosowane mechanizmy: Obsługa wyjątków w kodzie Java

The screenshot shows a web application window titled "Serwis rowerowy". The main content is a form titled "Dodaj Użytkownika". The form contains the following fields and values:

- Role: Serwisant (dropdown menu)
- Login: j.kowalski
- Hasło:
- Numer Telefonu: abcd
- Informacje dodatkowe: stażysta

Below the form is a button labeled "Dodaj Użytkownika". At the bottom left of the window is a button labeled "Powrót". A "Message" dialog box is open in the bottom right corner, displaying an information icon and the text "Niepoprawne dane!" with an "OK" button.

- b) Aplikacja jest zgodna ze scenariuszami jej funkcjonowania, między innymi dostępność funkcji jest zależna od roli pełnionej przez obecnie zalogowanego użytkownika.

Test uprawnień: Porównanie widoku Administratora i Serwisanta wykazało że użytkownik o niższych uprawnieniach nie ma dostępu do funkcji takich jak “Dodaj użytkownika”, “Usuń użytkownika” albo “Aktualizuj użytkownika”.

Weryfikacja: Przyciski te są całkowicie usuwane z interfejsu, co uniemożliwia wykonanie tych operacji.

Zgodność z logiką biznesową: Serwisant posiada dostęp tylko do modułów niezbędnych do jego pracy, co jest zgodne z założeniami projektu.

Widok administratora:



Widok serwisanta:



c) Wpływ aplikacji na zachowanie integralności bazy danych

Ochrona przed SQL Injection: w metodzie *zamknijZlecenie* zastosowano klasę *PreparedStatement*. Dzięki użyciu parametrów dane wejściowe traktowane są jako literały a nie fragmenty kodu SQL co eliminuje ryzyko ataku typu SQL Injection.

Hermetyzacja logiki: Aplikacja nie wykonuje bezpośrednich operacji na tabelach (np *update*). Zamiast tego wywołuje procedurę składowaną *call zamknij_zlecenie(?)*. Zapewnia to:

- Integralność: Baza danych sama dba o to aby przy zamykaniu zlecenia np. Zaktualizować datę zakończenia.
- Bezpieczeństwo: Użytkownik bazy, z której korzysta aplikacja, może mieć uprawnienia tylko do wykonywania konkretnych procedur a nie do bezpośredniej modyfikacji tabel.

```
public void zamknijZlecenie(int id) throws SQLException {  
    String sql = "call zamknij_zlecenie(?)";  
    PreparedStatement ps = conn.prepareStatement(sql);  
    ps.setInt(parameterIndex: 1, id);  
    ps.execute();  
    ps.close();  
}
```